

# Cónicas

April 30, 2026

## 1 Cónicas

Las explicaciones más detalladas de cada cónica están disponibles en [Apuntes Geometría Analítica](#)

### 1.1 Circunferencia

La circunferencia es el lugar geométrico de todos los puntos del plano, que equidistan de un punto fijo llamado centro ( $C$ ).

Toda circunferencia, con  $C(h, k)$  y radio ( $r$ ) se puede escribir como:

$$(x - h)^2 + (y - k)^2 = r^2$$

- Dicha fórmula se llama ecuación canónica de la circunferencia.

#### 1.1.1 Ecuación general de una circunferencia

$$x^2 + y^2 + Ax + By + C = 0$$

Código para representar una circunferencia

```
[ ]: import numpy as np
import matplotlib.pyplot as plt

def analizar_circunferencia():
    """
    Analiza y grafica una circunferencia desde su forma canónica o general.
    """
    print("=" * 50)
    print(" ANALIZADOR DE CIRCUNFERENCIAS")
    print("=" * 50)
    print("Seleccione el formato de entrada:")
    print("1. Forma Canónica: (x-h)^2 + (y-k)^2 = r^2")
    print("2. Forma General : x^2 + y^2 + Cx + Dy + E = 0")

    opcion = input("Ingrese 1 o 2: ").strip()

    if opcion == '1':
        print("\n--- Forma Canónica ---")
        h = float(input("Ingrese h (coordenada x del centro): "))
```

```

k = float(input("Ingrese k (coordenada y del centro): "))
r_cuadrado = float(input("Ingrese el radio al cuadrado (r^2): "))

if r_cuadrado <= 0:
    print("Error: El radio al cuadrado debe ser mayor a 0.")
    return
r = np.sqrt(r_cuadrado)

elif opcion == '2':
    print("\n--- Forma General ---")
    print("Nota: Se asume que los coeficientes de x^2 e y^2 son 1.")
    C = float(input("Ingrese C (coeficiente de x): "))
    D = float(input("Ingrese D (coeficiente de y): "))
    E = float(input("Ingrese E (término independiente): "))

    # Completación de cuadrados:  $(x + C/2)^2 + (y + D/2)^2 = (C/2)^2 + (D/2)^2 - E$ 
    h = -C / 2
    k = -D / 2
    r_cuadrado = h**2 + k**2 - E

    if r_cuadrado <= 0:
        print("Error: Los coeficientes forman un punto o un radio_
        ↪imaginario.")
        return
    r = np.sqrt(r_cuadrado)
    print(f"\n Ecuación canónica:  $(x - \{h:.2f\})^2 + (y - \{k:.2f\})^2 = \{r:.2f\}^2$ ")
    ↪2f}^2")

else:
    print("Opción no válida.")
    return

# --- ANÁLISIS ---
print("\n" + "-" * 50)
print(" RESULTADOS DEL ANÁLISIS")
print("-" * 50)
print(f"Centro (h,k) : ({h:.2f}, {k:.2f})")
print(f"Radio (r) : {r:.2f} unidades")
print(f"Diámetro : {2*r:.2f} unidades")
print("-" * 50)

# --- GRÁFICO ---
t = np.linspace(0, 2 * np.pi, 1000)
x_plot = h + r * np.cos(t)
y_plot = k + r * np.sin(t)

```

```

plt.figure(figsize=(8, 8))
plt.plot(x_plot, y_plot, label='Circunferencia', color='#2ca02c',
↪linewidth=2)
plt.plot(h, k, 'ko', label=f'Centro ({h:.2f}, {k:.2f})')

# Línea del radio
plt.plot([h, h+r], [k, k], 'r--', label=f'Radio = {r:.2f}')

plt.axhline(0, color='black', linewidth=1)
plt.axvline(0, color='black', linewidth=1)
plt.grid(color='gray', linestyle=':', alpha=0.7)
plt.axis('equal') # Vital para que sea redonda y no elíptica

plt.legend(loc='upper right')
plt.title('Representación Gráfica de la Circunferencia', fontweight='bold')
plt.xlabel('Eje X')
plt.ylabel('Eje Y')
plt.show()

analizar_circunferencia()

```

=====

#### ANALIZADOR DE CIRCUNFERENCIAS

=====

Seleccione el formato de entrada:

1. Forma Canónica:  $(x-h)^2 + (y-k)^2 = r^2$
2. Forma General :  $x^2 + y^2 + Cx + Dy + E = 0$

--- Forma General ---

Nota: Se asume que los coeficientes de  $x^2$  e  $y^2$  son 1.

Ecuación canónica:  $(x - -5.00)^2 + (y - 1.00)^2 = 3.00^2$

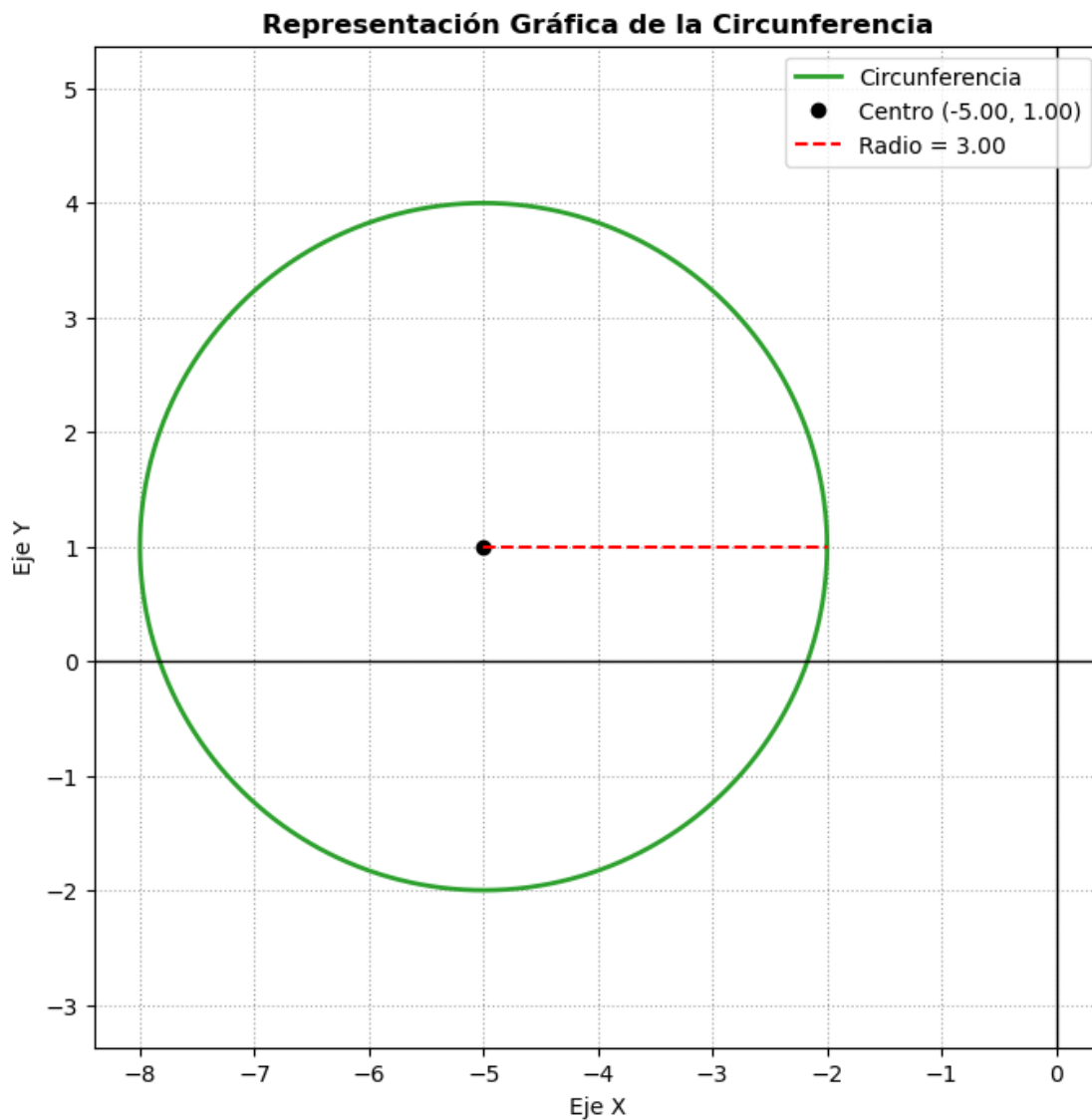
-----

#### RESULTADOS DEL ANÁLISIS

-----

Centro (h,k) : (-5.00, 1.00)  
Radio (r) : 3.00 unidades  
Diámetro : 6.00 unidades

-----



## 1.2 Parábola

Dada una recta  $L$  y  $F \notin L$ , una parábola es el lugar geométrico de todos los puntos  $P$  del plano que equidistan de  $F$  y de  $L$ .

- El punto  $F$  se llama **foco**.
- La recta  $L$  **directriz**.
- La recta perpendicular a  $L$  que pasa por  $F$  se llama **eje de simetría**.
- La intersección del eje de simetría con la parábola es el **vértice de la parábola**.

### 1.2.1 Ecuaciones canónicas

Caso 1

$$(x - h)^2 = 4p(y - k)$$

## Caso 2

$$(y - k)^2 = 4p(x - h)$$

### 1.2.2 Ecuación general de la parábola

$$Ax^2 + By^2 + Cx + Dy + E = 0$$

#### Código para representar una parábola

```
[2]: import numpy as np
import matplotlib.pyplot as plt

def analizar_parabola():
    """
    Analiza y grafica una parábola vertical u horizontal.
    """
    print("=" * 50)
    print(" ANALIZADOR DE PARÁBOLAS")
    print("=" * 50)
    print("Seleccione la orientación (basado en el término cuadrático):")
    print("1. Vertical (x-h)^2 = 4p(y-k) [o Ax^2 + Cx + Dy + E = 0]")
    print("2. Horizontal (y-k)^2 = 4p(x-h) [o By^2 + Cx + Dy + E = 0]")

    orientacion = input("Ingrese 1 o 2: ").strip()

    if orientacion not in ['1', '2']:
        print("Opción no válida.")
        return

    print("\nSeleccione el formato de entrada:")
    print("1. Forma Canónica")
    print("2. Forma General")
    formato = input("Ingrese 1 o 2: ").strip()

    if formato == '1':
        h = float(input("Ingrese h (coordenada x del vértice): "))
        k = float(input("Ingrese k (coordenada y del vértice): "))
        p = float(input("Ingrese el parámetro p (distancia vértice-foco): "))
        if p == 0:
            print("Error: El parámetro p no puede ser 0.")
            return

    elif formato == '2':
        if orientacion == '1': # Vertical (Ax^2 + Cx + Dy + E = 0)
            A = float(input("Ingrese A (coeficiente de x^2): "))
            C = float(input("Ingrese C (coeficiente de x): "))
            D = float(input("Ingrese D (coeficiente de y): "))
            E = float(input("Ingrese E (término independiente): "))
```

```

        if A == 0 or D == 0:
            print("Error: No es una parábola vertical válida (A y D no
↪ pueden ser 0).")
            return

        # Completación:  $A(x^2 + (C/A)x) = -Dy - E \rightarrow (x + C/2A)^2 = (-D/A)y$ 
↪ + ...
        h = -C / (2 * A)
        k = (C**2 / (4 * A) - E) / D
        p = -D / (4 * A)

    else: # Horizontal ( $By^2 + Cx + Dy + E = 0$ )
        B = float(input("Ingrese B (coeficiente de y^2): "))
        C = float(input("Ingrese C (coeficiente de x): "))
        D = float(input("Ingrese D (coeficiente de y): "))
        E = float(input("Ingrese E (término independiente): "))

        if B == 0 or C == 0:
            print("Error: No es una parábola horizontal válida (B y C no
↪ pueden ser 0).")
            return

        k = -D / (2 * B)
        h = (D**2 / (4 * B) - E) / C
        p = -C / (4 * B)

    print(f"\n Parámetros obtenidos: Vértice({h:.2f}, {k:.2f}), p={p:.2f}")

else:
    print("Formato no válido.")
    return

# --- ANÁLISIS ---
print("\n" + "-" * 50)
print(" RESULTADOS DEL ANÁLISIS")
print("-" * 50)
print(f"Vértice (h,k) : ({h:.2f}, {k:.2f})")
print(f"Parámetro (p) : {p:.2f}")
print(f"Lado Recto |4p|: {abs(4*p):.2f} unidades")

if orientacion == '1':
    print(f"Foco : ({h:.2f}, {k+p:.2f})")
    print(f"Directriz : Línea y = {k-p:.2f}")
    print(f"Eje de simetría: Línea x = {h:.2f}")
    apertura = "Arriba" if p > 0 else "Abajo"
else:

```

```

print(f"Foco          : ({h+p:.2f}, {k:.2f})")
print(f"Directriz     : Línea x = {h-p:.2f}")
print(f"Eje de simetría: Línea y = {k:.2f}")
apertura = "Derecha" if p > 0 else "Izquierda"

print(f"Apertura      : Hacia la {apertura}")
print("-" * 50)

# --- GRÁFICO ---
plt.figure(figsize=(8, 8))

if orientacion == '1': # Vertical
    # Rango de x para graficar (vértice +- 4 veces p para buena visibilidad)
    x_plot = np.linspace(h - abs(4*p), h + abs(4*p), 500)
    y_plot = (x_plot - h)**2 / (4 * p) + k
    plt.plot(x_plot, y_plot, label='Parábola', color='#ff7f0e', linewidth=2)
    plt.plot(h, k+p, 'gx', label='Foco', markersize=8) # Foco
    plt.axhline(k-p, color='r', linestyle='--', label='Directriz') #
    ↪Directriz

else: # Horizontal
    y_plot = np.linspace(k - abs(4*p), k + abs(4*p), 500)
    x_plot = (y_plot - k)**2 / (4 * p) + h
    plt.plot(x_plot, y_plot, label='Parábola', color='#ff7f0e', linewidth=2)
    plt.plot(h+p, k, 'gx', label='Foco', markersize=8) # Foco
    plt.axvline(h-p, color='r', linestyle='--', label='Directriz') #
    ↪Directriz

plt.plot(h, k, 'ko', label='Vértice') # Vértice general

plt.axhline(0, color='black', linewidth=1)
plt.axvline(0, color='black', linewidth=1)
plt.grid(color='gray', linestyle=':', alpha=0.7)
plt.axis('equal')
plt.legend()
plt.title('Representación Gráfica de la Parábola', fontweight='bold')
plt.show()

analizar_parabola()

```

# ANALIZADOR DE PARÁBOLAS

Seleccione la orientación (basado en el término cuadrático):

1. Vertical  $(x-h)^2 = 4p(y-k)$  [o  $Ax^2 + Cx + Dy + E = 0$ ]
2. Horizontal  $(y-k)^2 = 4p(x-h)$  [o  $By^2 + Cx + Dy + E = 0$ ]

Seleccione el formato de entrada:

1. Forma Canónica
2. Forma General

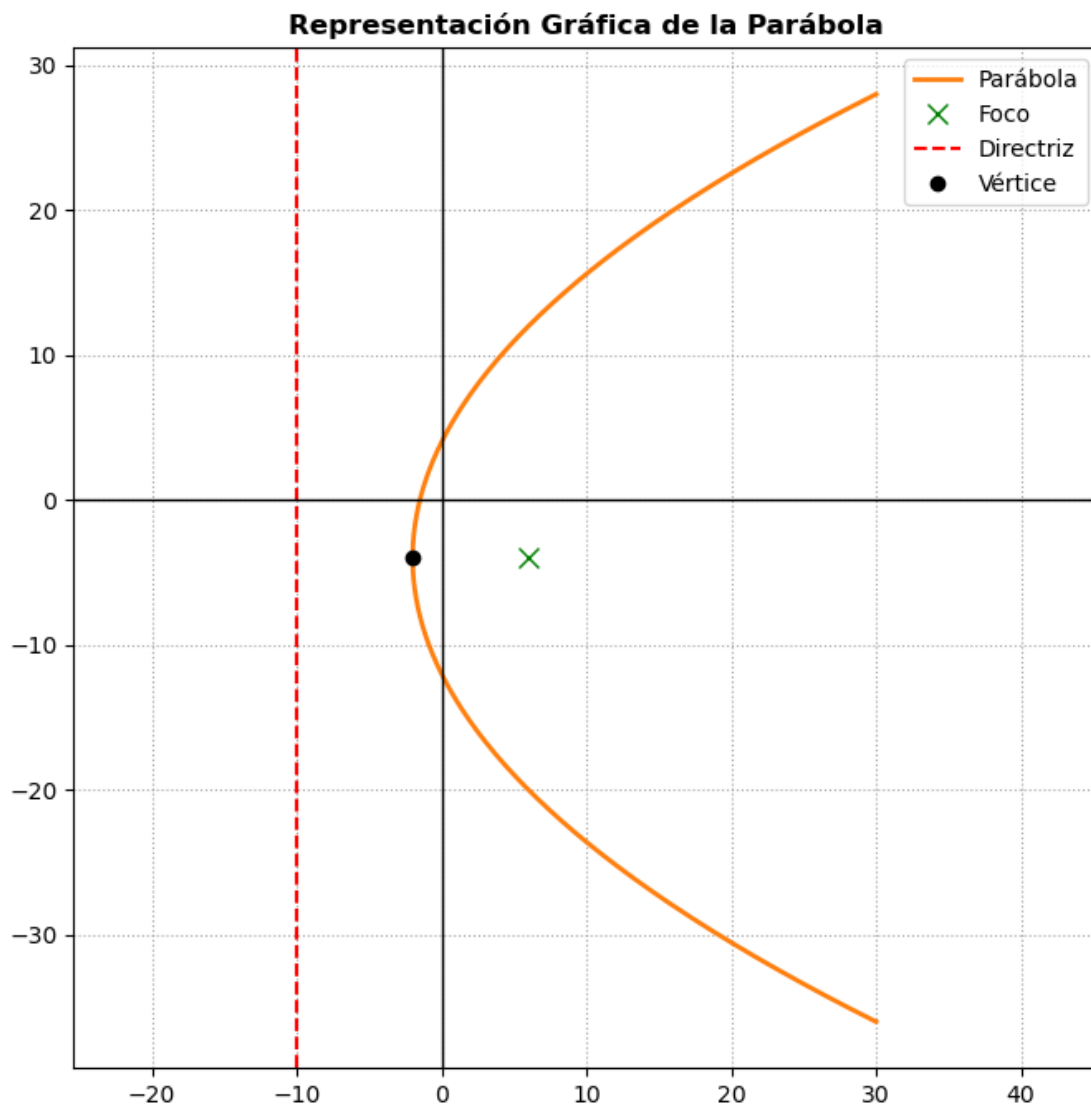
---

### RESULTADOS DEL ANÁLISIS

---

Vértice (h,k) : (-2.00, -4.00)  
Parámetro (p) : 8.00  
Lado Recto  $|4p|$ : 32.00 unidades  
Foco : (6.00, -4.00)  
Directriz : Línea  $x = -10.00$   
Eje de simetría: Línea  $y = -4.00$   
Apertura : Hacia la Derecha

---





### 1.3 Elipse

Una elipse es un lugar geometrico que se construlle con todos los puntoss  $P$ , tal que:

$$\forall P \in \text{Elipse} : d(P, F_1) + d(P + F_2) = k$$

Cuya forma general es:

$$Ax^2 + By^2 + Cx + Dy + E = 0$$

Aún más, según sea el caso, se tienen las ecuaciones canónicas de una elipse.

#### 1.3.1 Caso 1

$$\frac{(x-h)^2}{a^2} + \frac{(y-k)^2}{b^2} = 1$$

#### 1.3.2 Caso 2

$$\frac{(x-h)^2}{b^2} + \frac{(y-k)^2}{a^2} = 1$$

#### 1.3.3 Partes de una elipse

- Vértices.
- Eje mayor.
- Eje menor.
- Focos.
- Vértices.

Código para representar una elipse

```
[4]: import numpy as np
import matplotlib.pyplot as plt
import sys

def analizar_elipse():
    """
    Función interactiva para analizar y graficar una elipse
    a partir de su ecuación general o canónica.
    """
    print("=" * 50)
    print(" ANALIZADOR DE ELIPSES")
    print("=" * 50)
    print("Seleccione el formato de entrada:")
    print("1. Forma Canónica: (x-h)^2 / A_den + (y-k)^2 / B_den = 1")
    print("2. Forma General : Ax^2 + By^2 + Cx + Dy + E = 0")
```

```

opcion = input("Ingrese 1 o 2: ").strip()

# -----
# 1. INGRESO Y PROCESAMIENTO DE LA ECUACIÓN
# -----

if opcion == '1':
    print("\n--- Ha seleccionado la Forma Canónica ---")
    h = float(input("Ingrese el valor de h (coordenada x del centro): "))
    k = float(input("Ingrese el valor de k (coordenada y del centro): "))
    A_den = float(input("Ingrese el denominador debajo de las 'x' (A_den):"))
    B_den = float(input("Ingrese el denominador debajo de las 'y' (B_den):"))

    if A_den <= 0 or B_den <= 0:
        print("Error: Los denominadores deben ser estrictamente positivos.")
        return
    elif opcion == '2':
        print("\n--- Ha seleccionado la Forma General ---")
        A = float(input("Ingrese A (coeficiente de x^2): "))
        B = float(input("Ingrese B (coeficiente de y^2): "))
        C = float(input("Ingrese C (coeficiente de x): "))
        D = float(input("Ingrese D (coeficiente de y): "))
        E = float(input("Ingrese E (término independiente): "))
        # Validación matemática para elipses reales: A y B deben tener el mismo
        if A * B <= 0:
            print("Error: Los coeficientes A y B deben tener el mismo signo
            para ser una elipse.")
            return
        # Completación de cuadrados algebraicamente:
        #  $A(x^2 + (C/A)x) + B(y^2 + (D/B)y) = -E$ 
        # Se suma  $(C/2A)^2$  y  $(D/2B)^2$  compensando al otro lado
        h = -C / (2 * A)
        k = -D / (2 * B)
        # Término constante al lado derecho de la ecuación tras completar
        F = (C**2) / (4 * A) + (D**2) / (4 * B) - E
        if F <= 0:
            print("Error: Los coeficientes dados forman una elipse imaginaria o
            un solo punto.")
            return
        # Al dividir todo entre F, obtenemos los denominadores de la forma
        A_den = F / A

```

```

        B_den = F / B
        print(f"\n Ecuación canónica obtenida:")
        print(f"(x - {h:.2f})^2 / {A_den:.2f} + (y - {k:.2f})^2 / {B_den:.2f} = 1")
    else:
        print("Opción no válida. Ejecute el script nuevamente.")
        return

# -----
# 2. ANÁLISIS GEOMÉTRICO
# -----
# Determinamos la orientación evaluando cuál denominador es mayor
if A_den > B_den:
    orientacion = "Horizontal"
    a = np.sqrt(A_den) # Semieje mayor en X
    b = np.sqrt(B_den) # Semieje menor en Y
    c = np.sqrt(a**2 - b**2) # Distancia del centro al foco

    vertices = [(h - a, k), (h + a, k)]
    co_vertices = [(h, k - b), (h, k + b)]
    focos = [(h - c, k), (h + c, k)]
else:
    orientacion = "Vertical"
    a = np.sqrt(B_den) # Semieje mayor en Y
    b = np.sqrt(A_den) # Semieje menor en X
    c = np.sqrt(a**2 - b**2) # Distancia del centro al foco

    # Las coordenadas cambian de eje respecto al caso horizontal
    vertices = [(h, k - a), (h, k + a)]
    co_vertices = [(h - b, k), (h + b, k)]
    focos = [(h, k - c), (h, k + c)]

# -----
# 3. IMPRESIÓN DE RESULTADOS
# -----
print("\n" + "-" * 50)
print(" RESULTADOS DEL ANÁLISIS")
print("-" * 50)
print(f"Orientación : {orientacion}")
print(f"Centro (h,k) : ({h:.2f}, {k:.2f})")
print(f"Eje Mayor : {2*a:.2f} unidades")
print(f"Eje Menor : {2*b:.2f} unidades")
print(f"Dist. Focal : {2*c:.2f} unidades (Distancia entre los dos focos)")
print(f"Vértices : V1({vertices[0][0]:.2f}, {vertices[0][1]:.2f}) y V2({vertices[1][0]:.2f}, {vertices[1][1]:.2f})")
print(f"Co-vértices : C1({co_vertices[0][0]:.2f}, {co_vertices[0][1]:.2f}) y C2({co_vertices[1][0]:.2f}, {co_vertices[1][1]:.2f})")

```

```

print(f"Focos      : F1({focos[0][0]:.2f}, {focos[0][1]:.2f}) y_
↪F2({focos[1][0]:.2f}, {focos[1][1]:.2f})")
print("-" * 50)

# -----
# 4. GRÁFICO 2D
# -----
# Usamos ecuaciones paramétricas para dibujar la elipse:
#  $x(t) = h + a \cdot \cos(t)$ ,  $y(t) = k + b \cdot \sin(t)$ 
t = np.linspace(0, 2 * np.pi, 1000)
x_plot = h + np.sqrt(A_den) * np.cos(t)
y_plot = k + np.sqrt(B_den) * np.sin(t)

plt.figure(figsize=(8, 8))

# Dibujar la elipse
plt.plot(x_plot, y_plot, label='Elipse', color='#1f77b4', linewidth=2)

# Dibujar los puntos clave extrayendo las coordenadas X e Y de las listas
plt.plot(h, k, 'ko', label='Centro', markersize=6)

# Extraemos las 'x' y las 'y' de la lista de tuplas para los Vértices
v_x = [v[0] for v in vertices]
v_y = [v[1] for v in vertices]
plt.plot(v_x, v_y, 'ro', label='Vértices', markersize=6)

# Extraemos las 'x' y las 'y' de la lista de tuplas para los Focos
f_x = [f[0] for f in focos]
f_y = [f[1] for f in focos]
plt.plot(f_x, f_y, 'gx', label='Focos', markersize=7, marker='X')

# Dibujar los ejes mayor y menor de forma visual con líneas punteadas
plt.plot([v_x[0], v_x[1]], [v_y[0], v_y[1]], 'r--', alpha=0.5, label='Eje_
↪Mayor')
plt.plot([co_vertices[0][0], co_vertices[1][0]], [co_vertices[0][1],_
↪co_vertices[1][1]], 'b--', alpha=0.5, label='Eje Menor')

# Formato del plano cartesiano
plt.axhline(0, color='black', linewidth=1)
plt.axvline(0, color='black', linewidth=1)
plt.grid(color='gray', linestyle=':', alpha=0.7)

# IMPORTANTE: Forzamos la relación de aspecto 1:1 para que la elipse no se_
↪deforme
plt.axis('equal')

plt.legend(loc='upper right')

```

```

plt.title('Representación Gráfica de la Elipse', fontsize=14,
fontweight='bold')
plt.xlabel('Eje X', fontsize=12)
plt.ylabel('Eje Y', fontsize=12)

# Mostrar gráfico
plt.show()

analizar_elipse()

```

```

=====
ANALIZADOR DE ELIPSES
=====
Seleccione el formato de entrada:
1. Forma Canónica:  $(x-h)^2 / A_{den} + (y-k)^2 / B_{den} = 1$ 
2. Forma General :  $Ax^2 + By^2 + Cx + Dy + E = 0$ 

```

--- Ha seleccionado la Forma Canónica ---

```

-----
RESULTADOS DEL ANÁLISIS
-----
Orientación   : Horizontal
Centro (h,k)  : (2.00, 5.00)
Eje Mayor     : 10.00 unidades
Eje Menor     : 6.00 unidades
Dist. Focal   : 8.00 unidades (Distancia entre los dos focos)
Vértices      : V1(-3.00, 5.00) y V2(7.00, 5.00)
Co-vértices   : C1(2.00, 2.00) y C2(2.00, 8.00)
Focos         : F1(-2.00, 5.00) y F2(6.00, 5.00)

```

```

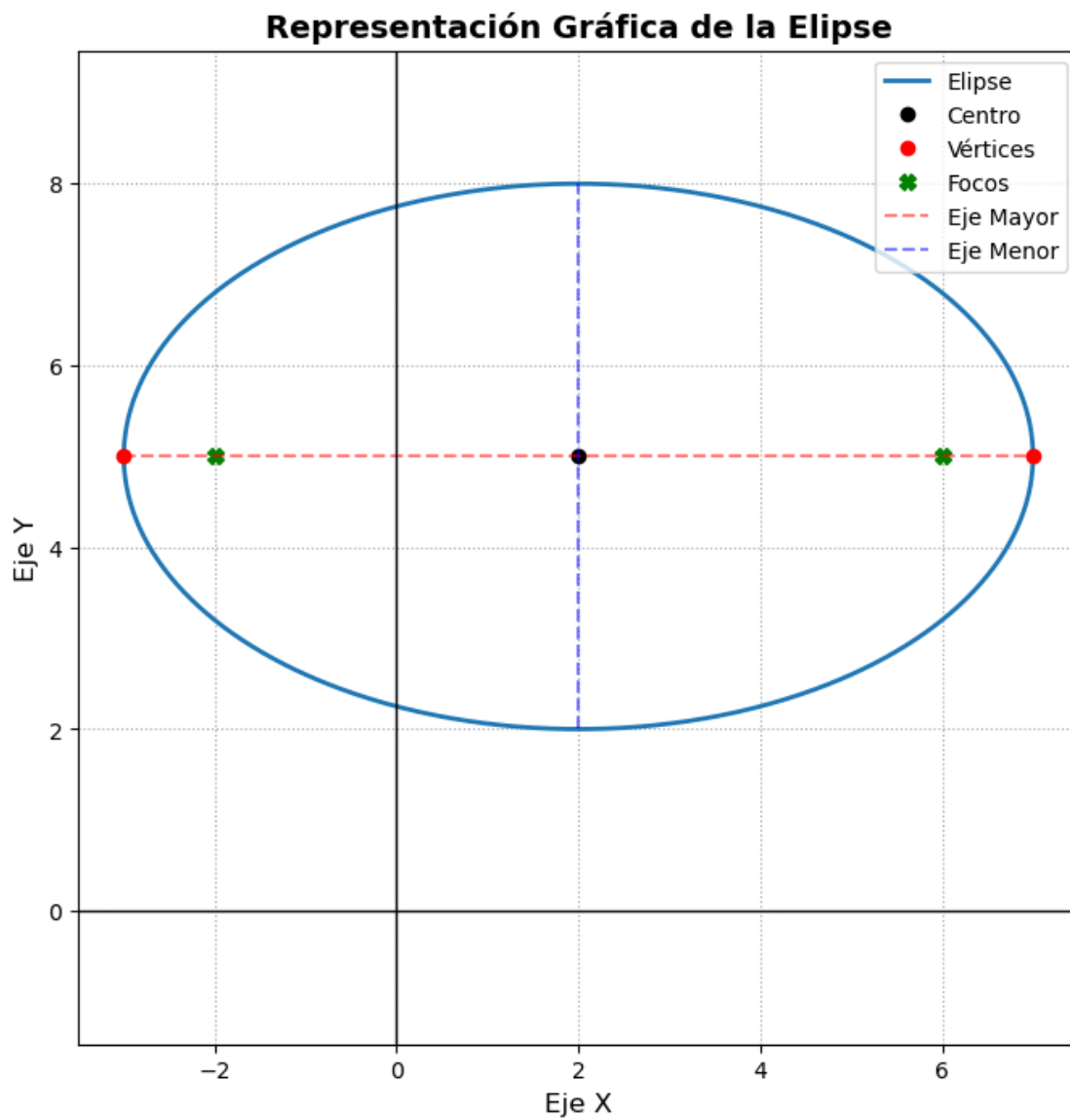
/tmp/ipykernel_57482/3927223867.py:126: UserWarning: marker is redundantly
defined by the 'marker' keyword argument and the fmt string "gx" (->
marker='x'). The keyword argument will take precedence.

```

```

plt.plot(f_x, f_y, 'gx', label='Focos', markersize=7, marker='X')

```



## 1.4 Hipérbola

Una hipérbola se construye con todos los puntos  $P$ , tal que:

$$|d(P, F_1) - d(P, F_2)| = k$$

### 1.4.1 Ecuaciones canónicas

**Caso 1**

$$\frac{(x - h)^2}{a^2} - \frac{(y - k)^2}{b^2} = 1$$

## Caso 2

$$\frac{(y-k)^2}{a^2} - \frac{(x-h)^2}{b^2} = 1$$

### 1.4.2 Ecuación general de la hipérbola

$$Ax^2 + By^2 + Cx + Dy + E = 0$$

#### Código para representar una hipérbola

```
[5]: import numpy as np
import matplotlib.pyplot as plt

def analizar_hiperbola():
    """
    Analiza y grafica una hipérbola transformando ecuaciones algebraicamente.
    """
    print("=" * 50)
    print(" ANALIZADOR DE HIPÉRBOLAS")
    print("=" * 50)
    print("Seleccione el formato de entrada:")
    print("1. Forma Canónica")
    print("2. Forma General : Ax^2 + By^2 + Cx + Dy + E = 0")

    opcion = input("Ingrese 1 o 2: ").strip()

    if opcion == '1':
        print("\nOrientación: 1. Horizontal (x positivo) | 2. Vertical (y_
        ↪positivo)")
        ori = input("Ingrese 1 o 2: ").strip()
        h = float(input("Ingrese h (x del centro): "))
        k = float(input("Ingrese k (y del centro): "))
        a_cuadrado = float(input("Ingrese a^2 (denominador del término_
        ↪positivo): "))
        b_cuadrado = float(input("Ingrese b^2 (denominador del término_
        ↪negativo): "))

        a = np.sqrt(a_cuadrado)
        b = np.sqrt(b_cuadrado)
        orientacion = "Horizontal" if ori == '1' else "Vertical"

    elif opcion == '2':
        A = float(input("Ingrese A (coeficiente de x^2): "))
        B = float(input("Ingrese B (coeficiente de y^2): "))
        C = float(input("Ingrese C (coeficiente de x): "))
        D = float(input("Ingrese D (coeficiente de y): "))
        E = float(input("Ingrese E (término independiente): "))

        if A * B >= 0:
```

```

        print("Error: En una hipérbola, A y B deben tener signos opuestos.")
        return

    # Completación de cuadrados
    h = -C / (2 * A)
    k = -D / (2 * B)
    F = (C**2) / (4 * A) + (D**2) / (4 * B) - E

    if F == 0:
        print("Error: Hipérbola degenerada (son dos líneas rectas secantes).")
        return

    # Denominadores: F/A y F/B. Uno será positivo y el otro negativo.
    den_X = F / A
    den_Y = F / B

    if den_X > 0:
        orientacion = "Horizontal"
        a = np.sqrt(den_X)
        b = np.sqrt(abs(den_Y))
    else:
        orientacion = "Vertical"
        a = np.sqrt(den_Y)
        b = np.sqrt(abs(den_X))

    print(f"\n Ecuación canónica procesada (Centro en {h:.2f}, {k:.2f})")

else:
    return

# --- ANÁLISIS GEOMÉTRICO ---
c = np.sqrt(a**2 + b**2) # Pitágoras para la hipérbola (se suman los
cuadrados)

if orientacion == "Horizontal":
    vertices = [(h - a, k), (h + a, k)]
    focos = [(h - c, k), (h + c, k)]
    asintota1 = fr"y - {k:.2f} = {b/a:.2f}(x - {h:.2f})"
    asintota2 = fr"y - {k:.2f} = {-b/a:.2f}(x - {h:.2f})"
    m_asintota = b/a
else:
    vertices = [(h, k - a), (h, k + a)]
    focos = [(h, k - c), (h, k + c)]
    asintota1 = fr"y - {k:.2f} = {a/b:.2f}(x - {h:.2f})"
    asintota2 = fr"y - {k:.2f} = {-a/b:.2f}(x - {h:.2f})"
    m_asintota = a/b

```



```

print("\n" + "-" * 50)
print(" RESULTADOS DEL ANÁLISIS")
print("-" * 50)
print(f"Orientación      : {orientacion}")
print(f"Centro (h,k)      : ({h:.2f}, {k:.2f})")
print(f"Eje Transverso 2a: {2*a:.2f} (Distancia entre vértices)")
print(f"Eje Conjugado 2b : {2*b:.2f}")
print(f"Dist. Focal 2c    : {2*c:.2f} (Distancia entre focos)")
print(f"Vértices          : V1({vertices[0][0]:.2f}, {vertices[0][1]:.2f}) y
↪V2({vertices[1][0]:.2f}, {vertices[1][1]:.2f})")
print(f"Focos           : F1({focos[0][0]:.2f}, {focos[0][1]:.2f}) y
↪F2({focos[1][0]:.2f}, {focos[1][1]:.2f})")
print(f"Asíntota 1      : {asintota1}")
print(f"Asíntota 2      : {asintota2}")
print("-" * 50)

# --- GRÁFICO ---
plt.figure(figsize=(8, 8))

# Parámetro t para la curva paramétrica usando funciones hiperbólicas
↪(cosh, sinh)
t = np.linspace(-2.5, 2.5, 500)

if orientacion == "Horizontal":
    # Rama derecha e izquierda
    x_rama1 = h + a * np.cosh(t)
    x_rama2 = h - a * np.cosh(t)
    y_ramas = k + b * np.sinh(t)
    plt.plot(x_rama1, y_ramas, color='#d62728', linewidth=2,
↪label='Hipérbola')
    plt.plot(x_rama2, y_ramas, color='#d62728', linewidth=2)

    # Asíntotas
    x_asintota = np.linspace(h - a*4, h + a*4, 100)
    y_asintota1 = k + m_asintota * (x_asintota - h)
    y_asintota2 = k - m_asintota * (x_asintota - h)

else:
    # Rama superior e inferior
    x_ramas = h + b * np.sinh(t)
    y_rama1 = k + a * np.cosh(t)
    y_rama2 = k - a * np.cosh(t)
    plt.plot(x_ramas, y_rama1, color='#d62728', linewidth=2,
↪label='Hipérbola')
    plt.plot(x_ramas, y_rama2, color='#d62728', linewidth=2)

```

```

# Asíntotas
x_asintota = np.linspace(h - b*4, h + b*4, 100)
y_asintota1 = k + m_asintota * (x_asintota - h)
y_asintota2 = k - m_asintota * (x_asintota - h)

# Dibujo de Asíntotas
plt.plot(x_asintota, y_asintota1, 'b--', alpha=0.5, label='Asíntotas')
plt.plot(x_asintota, y_asintota2, 'b--', alpha=0.5)

# Puntos clave
plt.plot(h, k, 'ko', label='Centro')
plt.plot([v[0] for v in vertices], [v[1] for v in vertices], 'ro',
↪label='Vértices')
plt.plot([f[0] for f in focos], [f[1] for f in focos], 'gx', markersize=8,
↪label='Focos')

plt.axhline(0, color='black', linewidth=1)
plt.axvline(0, color='black', linewidth=1)
plt.grid(color='gray', linestyle=':', alpha=0.7)
plt.axis('equal')
plt.legend()
plt.title('Representación Gráfica de la Hipérbola', fontweight='bold')
plt.show()

analizar_hiperbola()

```

```

=====
ANALIZADOR DE HIPÉRBOLAS
=====
Seleccione el formato de entrada:
1. Forma Canónica
2. Forma General :  $Ax^2 + By^2 + Cx + Dy + E = 0$ 

Ecuación canónica procesada (Centro en 1.00, -2.00)

-----
RESULTADOS DEL ANÁLISIS
-----
Orientación      : Horizontal
Centro (h,k)     : (1.00, -2.00)
Eje Transverso 2a: 2.00 (Distancia entre vértices)
Eje Conjugado 2b : 4.00
Dist. Focal 2c   : 4.47 (Distancia entre focos)
Vértices         : V1(0.00, -2.00) y V2(2.00, -2.00)
Focos            : F1(-1.24, -2.00) y F2(3.24, -2.00)
Asíntota 1       :  $y - -2.00 = 2.00(x - 1.00)$ 

```

Asíntota 2 :  $y - -2.00 = -2.00(x - 1.00)$

---

